

CSE420

Compiler Design

Syntax Analysis

Context-Free Grammars, Derivations, Parse Trees, Ambiguity, Bottom-Up & Shift-Reduce Parsing

Prepared by Azmari Sultana

Table of Contents

1. Introduction to Syntax Analysis.....	3
1.1 Why Grammars Are Used for Syntax Specification.....	3
1.2 The Role of the Parser.....	3
1.3 Types of Parsers.....	3
2. Context-Free Grammars (CFG).....	4
2.1 Notational Conventions.....	4
2.2 Standard Expression Grammar (4.1) — Very Important.....	5
3. Derivations.....	5
3.1 Leftmost and Rightmost Derivations.....	5
4. Parse Trees.....	6
5. Ambiguity.....	7
6. Verifying the Language Generated by a Grammar.....	7
7. Syntax Error Handling.....	8
7.1 Levels of Errors.....	8
7.2 Goals of an Error Handler.....	8
7.3 Viable-Prefix Property.....	8
7.4 Error-Recovery Strategies.....	8
8. Bottom-Up Parsing.....	9
8.1 Reductions.....	9
8.2 Handle Pruning.....	9
9. Shift-Reduce Parsing — Mechanics.....	10
9.1 Four Possible Actions.....	10
9.2 General Shift-Reduce Parsing Cycle.....	11
10. Worked Examples — Step-by-Step Simulations.....	11
Example 1: Parse the string $id_1 * id_2$	11
Example 2: Parse the string $(id) + id$	12
11. Conflicts During Shift-Reduce Parsing.....	13
11.1 Shift/Reduce Conflict.....	13
11.2 Reduce/Reduce Conflict.....	13
12. Practice Exercises (Textbook Section 4.5.5 Style).....	14
Exercise A — Identify the Handle.....	14
Exercise B — Full Bottom-Up (Shift-Reduce) Parses.....	14

1. Introduction to Syntax Analysis

Syntax analysis is the second phase of a compiler, coming right after lexical analysis. It checks whether the sequence of tokens produced by the lexical analyzer follows the grammatical (structural) rules of the programming language.

Syntax analysis is also called parsing, and the module that performs it is called the parser (or syntax analyzer).

1.1 Why Grammars Are Used for Syntax Specification

- Provide an easy-to-understand, precise specification of syntax.
- Parsers can often be built automatically from a grammar (like lexical analyzers are built from regular expressions).
- Grammars make it easy to extend a language with new constructs.
- Grammar-based translation allows the program to be converted into an intermediate representation useful for later compiler phases.

Theory Questions — Introduction

Q: What is syntax analysis?

Ans: Syntax analysis is the phase of compilation that verifies whether a string of tokens (from the lexer) can be generated by the grammar of the source language, i.e., whether the program is structurally/grammatically valid.

Q: Why is syntax analysis called parsing?

Ans: Because it breaks the program down into its constituent parts (like parsing an essay into words/phrases) and checks how those parts combine to form the whole program, similar to grammatical parsing in natural languages.

Q: What is the difference between parsing a natural language essay and parsing a program?

Ans: In natural language, grammar only concerns individual sentence structure — we never say the whole essay is 'ungrammatical', only that specific sentences have errors. In programming languages, grammar validates both individual parts (statements/expressions) AND the program as a whole.

Q: What are the benefits of using grammars in compiler design?

Ans: (1) Easy, precise syntax specification, (2) Automatic parser construction, (3) Easy language evolution by adding new grammar rules, (4) Enables translation to intermediate code and error detection.

Q: Why is a context-free grammar used for programming languages instead of a context-sensitive one?

Ans: Because programming language constructs (like assignment statements) do not need surrounding context to be validated — a rule can be checked purely by its own structure. This keeps grammars simple and parsers efficient, unlike natural languages where meaning often depends on context (e.g., resolving a pronoun like 'it').

1.2 The Role of the Parser

The parser gets a stream of tokens from the lexical analyzer (calling 'get next token' as needed) and checks if the token stream can be generated by the grammar. It reports syntax errors clearly and tries to recover so that parsing can continue. Conceptually, the parser builds a parse tree, but in practice this tree may not be built explicitly — translation actions can happen alongside parsing.

Pipeline: Source Program → Lexical Analyzer → Parser → Rest of Front End → Intermediate Representation. The Parser and Lexical Analyzer both interact with the Symbol Table.

1.3 Types of Parsers

- Universal parsing methods (e.g., CYK algorithm, Earley's algorithm): can parse any grammar, but too inefficient for production compilers.
- Top-down parsing: builds the parse tree from the root down to the leaves. Works for LL grammars.

- Bottom-up parsing: builds the parse tree from the leaves up to the root. Works for the larger class of LR grammars.

In both top-down and bottom-up parsing, input is scanned left to right, one symbol at a time.

Theory Questions — Role & Types of Parsers

Q: What is the role of the parser in a compiler?

Ans: It obtains tokens from the lexical analyzer, verifies the token stream matches the grammar, reports syntax errors, recovers from common errors, and (conceptually) constructs a parse tree passed to the rest of the front end.

Q: Why are universal parsing methods not used in production compilers?

Ans: Because although they can parse any grammar, they are computationally too inefficient (e.g., CYK and Earley's algorithms) for practical, fast compilation.

Q: What is the difference between top-down and bottom-up parsing?

Ans: Top-down parsing builds the parse tree starting from the root (start symbol) and expands down to the leaves (tokens), predicting productions. Bottom-up parsing starts from the leaves (tokens) and combines/reduces them upward until the root (start symbol) is reached.

Q: Why do LL parsers use top-down parsing and LR parsers use bottom-up parsing?

Ans: By definition/design: LL grammars support predictive, left-to-right, leftmost-derivation construction (top-down), while LR grammars support left-to-right, rightmost-derivation-in-reverse construction (bottom-up), which is more powerful and covers more grammars.

2. Context-Free Grammars (CFG)

A context-free grammar consists of four components:

- Terminals — basic symbols (tokens) from which strings are formed. Cannot be broken down further.
- Nonterminals — syntactic variables denoting sets of strings; they impose hierarchical structure.
- Start Symbol — one nonterminal designated to represent the whole language generated by the grammar.
- Productions — rules of the form Head $\rightarrow$ Body, specifying how a nonterminal can be replaced by a string of terminals/nonterminals.

Example: $\text{stmt} \rightarrow \text{if (expr) stmt else stmt}$. Here 'if', 'else', '(', ')' are terminals; stmt and expr are nonterminals.

2.1 Notational Conventions

- Lowercase letters early in alphabet (a, b, c) = terminals
- Operator/punctuation symbols (+, *, (,), comma) = terminals
- Digits 0-9 = terminals
- Boldface strings (id, if) = single terminal symbols
- Uppercase letters early in alphabet (A, B, C) = nonterminals
- S = usually the start symbol
- Lowercase italic names (expr, stmt) = nonterminals
- E, T, F = commonly used for Expression, Term, Factor nonterminals
- Uppercase letters late in alphabet (X, Y, Z) = grammar symbols (terminal OR nonterminal)
- Lowercase letters late in alphabet (u...z) = strings of terminals
- Greek letters (α , β , γ) = strings of grammar symbols (may include nonterminals)
- $A \rightarrow \alpha_1 \mid \alpha_2 \mid \dots \mid \alpha_k$ means multiple alternative bodies (called alternatives) for head A
- Unless stated otherwise, the head of the FIRST production is the start symbol

2.2 Standard Expression Grammar (4.1) — Very Important

$$E \rightarrow E + T \mid T \quad T \rightarrow T * F \mid F \quad F \rightarrow (E) \mid \text{id}$$

This is an LR grammar (suitable for bottom-up parsing) but is left-recursive, so it CANNOT be used for top-down parsing.

Non-left-recursive version (used for top-down/predictive parsing) — Grammar (4.2)

$$E \rightarrow T E' \quad E' \rightarrow + T E' \mid \epsilon \quad T \rightarrow F T' \quad T' \rightarrow * F T' \mid \epsilon \quad F \rightarrow (E) \mid \text{id}$$

Ambiguous version — Grammar (4.3)

$$E \rightarrow E + E \mid E * E \mid (E) \mid \text{id}$$

This treats + and * alike and allows more than one parse tree for expressions like $a + b * c$ — hence it is ambiguous.

Theory Questions — CFG

Q: What are the four components of a context-free grammar?

Ans: Terminals, Nonterminals, a Start Symbol, and Productions (rules).

Q: What is a terminal? What is a nonterminal?

Ans: A terminal is a basic symbol (token) of the language that appears in the final string and cannot be replaced further. A nonterminal is a syntactic variable representing a set of strings; it can be replaced/expanded using productions.

Q: Why is expression grammar (4.1) unsuitable for top-down parsing?

Ans: Because it is left-recursive ($E \rightarrow E + T$). A top-down/predictive parser would recurse infinitely trying to expand E without consuming any input, causing an infinite loop.

Q: Why is grammar (4.3) ambiguous?

Ans: Because it allows the same string, e.g., $\text{id} + \text{id} * \text{id}$, to be derived by more than one leftmost (or rightmost) derivation / to have more than one parse tree, since it does not encode operator precedence or associativity.

Q: What is the significance of the start symbol?

Ans: The start symbol represents the entire language generated by the grammar; parsing succeeds only when the input string can be derived starting from this symbol.

3. Derivations

A derivation is a sequence of replacement steps, starting from the start symbol, where at each step a nonterminal is replaced by the body of one of its productions.

- $\alpha \Rightarrow \beta$ means ' α derives β in one step'.
- $\alpha \Rightarrow^* \beta$ means ' α derives β in zero or more steps'.
- $\alpha \Rightarrow^+ \beta$ means ' α derives β in one or more steps'.
- Sentential form: any string α such that $S \Rightarrow^* \alpha$ (may contain terminals and nonterminals).
- Sentence: a sentential form with NO nonterminals (only terminals).
- Language generated by grammar G , $L(G)$: the set of all sentences of G .

3.1 Leftmost and Rightmost Derivations

- Leftmost derivation: at every step, the LEFTMOST nonterminal is replaced. Written $\alpha \Rightarrow_{lm} \beta$.
- Rightmost derivation (canonical derivation): at every step, the RIGHTMOST nonterminal is replaced. Written $\alpha \Rightarrow_{rm} \beta$.

Example (grammar $E \rightarrow E + E \mid E * E \mid -E \mid (E) \mid \text{id}$), deriving $-(\text{id} + \text{id})$:

Leftmost: $E \Rightarrow -E \Rightarrow -(E) \Rightarrow -(E+E) \Rightarrow -(id+E) \Rightarrow -(id+id)$ Rightmost: $E \Rightarrow -E \Rightarrow -(E) \Rightarrow -(E+E) \Rightarrow -(E+id) \Rightarrow -(id+id)$

Bottom-up parsing constructs a rightmost derivation, but builds it in REVERSE order (from the sentence back to the start symbol).

Theory Questions — Derivations

Q: What is a derivation?

Ans: A sequence of production applications, starting from the start symbol, that rewrites nonterminals step by step until (optionally) only terminals remain.

Q: What is the difference between a sentential form and a sentence?

Ans: A sentential form may contain both terminals and nonterminals (any string derivable from S). A sentence is a sentential form containing ONLY terminals — i.e., a fully derived string.

Q: What is the difference between leftmost and rightmost derivation?

Ans: In a leftmost derivation, the leftmost nonterminal is expanded at every step. In a rightmost derivation, the rightmost nonterminal is expanded at every step. Both can lead to the same parse tree but via different orders of replacement.

Q: Why does bottom-up parsing relate to rightmost derivation in reverse?

Ans: Because bottom-up parsing builds the parse tree from leaves to root; reversing this process (root to leaves) corresponds exactly to a rightmost derivation, so bottom-up parsing is said to trace out a rightmost derivation in reverse.

Q: What is a left-sentential form and a right-sentential form?

Ans: A left-sentential form is any sentential form that appears in some leftmost derivation from S. A right-sentential form is any sentential form appearing in some rightmost derivation from S.

4. Parse Trees

A parse tree is a graphical representation of a derivation that filters out the ORDER in which productions were applied — many different derivation orders can correspond to the same parse tree.

- Each interior node = application of a production; labeled with the head nonterminal.
- Children of a node = symbols in the body of that production, left to right.
- Leaves = terminals or nonterminals; read left to right, they form the yield (or frontier) of the tree.

Important fact: Every parse tree has exactly ONE unique leftmost derivation and exactly ONE unique rightmost derivation associated with it (even though many different derivation orders can build the same tree).

Theory Questions — Parse Trees

Q: What is a parse tree?

Ans: A tree representation showing how a start symbol derives a given string, where each internal node represents a production applied and children represent the production's body symbols.

Q: What is the yield/frontier of a parse tree?

Ans: The sequence of leaf node labels read left to right — this equals the string (sentential form) derived by the tree.

Q: Why is there a many-to-one relationship between derivations and parse trees?

Ans: Because a parse tree doesn't record the ORDER in which nonterminals were replaced — only WHICH productions were used and WHERE. Different orders of applying the same productions can build the identical tree.

Q: How many leftmost/rightmost derivations correspond to one parse tree?

Ans: Exactly one unique leftmost derivation and exactly one unique rightmost derivation correspond to any given parse tree.

5. Ambiguity

A grammar is ambiguous if it produces more than one parse tree for some sentence — equivalently, if it has more than one leftmost derivation OR more than one rightmost derivation for the same string.

Example: Grammar $E \rightarrow E+E \mid E^*E \mid (E) \mid \text{id}$ has TWO leftmost derivations for $\text{id}+\text{id}*\text{id}$, producing two different parse trees (one treats $+$ before $*$, the other $*$ before $+$).

Since an ambiguous grammar does not let us uniquely pick a parse tree, it is generally undesirable for compilers.

Solutions:

- Rewrite the grammar to be unambiguous (e.g., using separate E, T, F levels to encode precedence and left-associativity — see Grammar 4.1).
- Keep the ambiguous grammar but add disambiguating rules (precedence/associativity declarations) that tell the parser which parse tree to prefer.

Theory Questions — Ambiguity

Q: What is an ambiguous grammar? How do you prove a grammar is ambiguous?

Ans: A grammar is ambiguous if some sentence has more than one parse tree (equivalently more than one leftmost or rightmost derivation). To prove ambiguity, find one input string and show two distinct derivations/parse trees for it.

Q: Why must a grammar for a programming language usually be unambiguous?

Ans: Because an ambiguous grammar cannot uniquely determine the meaning/structure (parse tree) of a program, so the compiler cannot know exactly how to translate it. Different parse trees can imply different semantics (e.g., different operator evaluation order).

Q: How can ambiguity be resolved without rewriting the grammar?

Ans: By using disambiguating rules such as precedence and associativity declarations, which tell the parser to prefer one parse tree/derivation over another (e.g., resolving the dangling-else by preferring to shift 'else').

Q: Give an example of ambiguity from Grammar (4.3).

Ans: The string $\text{id}+\text{id}*\text{id}$ has two leftmost derivations under $E \rightarrow E+E \mid E^*E \mid \text{id}$ — one where $+$ is applied first and one where $*$ is applied first — giving different associations of operators, hence different parse trees.

Q: What is the classic example of an ambiguous grammar in control structures?

Ans: The dangling-else grammar: $\text{stmt} \rightarrow \text{if expr then stmt} \mid \text{if expr then stmt else stmt} \mid \text{other}$. It is ambiguous because after 'if expr then stmt', on seeing 'else' the parser cannot decide (from the grammar alone) which 'if' the else belongs to.

6. Verifying the Language Generated by a Grammar

Sometimes we must prove that a given grammar generates exactly a particular language L . A proof has two parts:

- (1) Show every string generated by grammar G is in L .
- (2) Show every string in L can indeed be generated by G .

Example: the grammar $S \rightarrow (S)S \mid \epsilon$ generates exactly all strings of balanced parentheses (and nothing else). This is proved using induction on the number of derivation steps:

- Basis: the shortest derivation (1 step) gives the empty string ϵ , which is trivially balanced.

- Induction: assume all derivations of fewer than n steps give balanced strings. A leftmost derivation of exactly n steps has the form $S \Rightarrow (S)S \Rightarrow^* (x)y$, where x and y are each derived in fewer than n steps, so by the inductive hypothesis both x and y are balanced — therefore $(x)y$ is also balanced.

Theory Questions — Verifying Grammars

Q: What two things must be proven to show a grammar G generates a language L ?

Ans: (1) Every string derivable from G 's start symbol is in L , and (2) every string in L can be derived from G 's start symbol. Both directions are required for a complete proof.

Q: What grammar generates the language of all strings of balanced parentheses?

Ans: $S \rightarrow (S)S \mid \epsilon$ — this generates exactly all balanced-parentheses strings and nothing else.

Q: What proof technique is typically used for such grammar-language proofs?

Ans: Mathematical induction — usually on the number of steps in a derivation (or on the height of the parse tree), with a base case (shortest derivation) and an inductive step assuming the property holds for all shorter derivations.

7. Syntax Error Handling

7.1 Levels of Errors

- Lexical errors: misspelled identifiers/keywords, missing quotes, etc.
- Syntactic errors: misplaced semicolons, missing/extra braces, wrongly structured statements.
- Semantic errors: type mismatches between operators and operands.
- Logical errors: incorrect reasoning, e.g., using $=$ instead of $==$ (still grammatically valid but wrong intent).

7.2 Goals of an Error Handler

- Report the presence of errors clearly and accurately.
- Recover quickly enough to detect subsequent errors.
- Add minimal overhead to processing correct programs.

7.3 Viable-Prefix Property

LL and LR parsing methods detect an error as soon as possible — as soon as the token stream seen so far (prefix) cannot possibly be extended to a valid string in the language. This is called the viable-prefix property.

7.4 Error-Recovery Strategies

- Panic-Mode Recovery: discard input tokens one at a time until a synchronizing token (like $;$ or $\}$) is found. Simple, guaranteed not to loop infinitely, but skips input without checking for further errors.
- Phrase-Level Recovery: perform local correction on the remaining input (e.g., replace comma with semicolon, insert a missing semicolon). Can correct any input, but struggles if the true error occurred before the detection point.
- Error Productions: augment the grammar with extra productions that match commonly made mistakes, so the parser can directly recognize and report anticipated errors.
- Global Correction: algorithmically finds the minimal number of insertions/deletions/changes to transform the wrong input into a correct one. Theoretically ideal but too costly (time/space) for real compilers.

Theory Questions — Error Handling

Q: What are the four levels of programming errors?

Ans: Lexical, Syntactic, Semantic, and Logical errors.

Q: What is the viable-prefix property?

Ans: The property of LL/LR parsers whereby they can detect an error as soon as they read a prefix of input that cannot be completed to form any valid string in the language — i.e., errors are detected as early as possible.

Q: What is panic-mode recovery? What is its main advantage and disadvantage?

Ans: On error, the parser discards tokens until it finds a synchronizing token (like ; or }). Advantage: simple, never loops infinitely. Disadvantage: it may skip a large amount of input without checking it for more errors.

Q: What is phrase-level recovery? What is its drawback?

Ans: The parser locally corrects the input around the error point (e.g., insert/delete/replace a token) to continue parsing. Drawback: it struggles when the real error actually happened before the point where it was detected.

Q: Why is global correction rarely used in practice?

Ans: Because although it finds the theoretically minimal-cost correction (fewest insertions/deletions/changes), the algorithms are too computationally expensive (time and space) for production compilers.

Q: What are error productions?

Ans: Productions deliberately added to a grammar that match known/anticipated erroneous constructs, letting the parser detect and report a specific, expected mistake directly.

8. Bottom-Up Parsing

A bottom-up parse builds the parse tree starting at the leaves (bottom) and working up towards the root (top) — the reverse of top-down parsing. Bottom-up parsing is also called SHIFT-REDUCE PARSING because its two main actions are shift and reduce.

8.1 Reductions

Bottom-up parsing can be thought of as 'reducing' a string w down to the start symbol of the grammar. At each reduction step, a specific substring matching the BODY of a production is replaced by the NONTERMINAL at the HEAD of that production — this is the exact reverse of a derivation step.

Example: Parsing $id * id$ using grammar (4.1). Reduction sequence:

$id * id \rightarrow F * id \rightarrow T * id \rightarrow T * F \rightarrow T \rightarrow E$

Corresponding rightmost derivation (read in reverse of the reduction sequence):

$E \Rightarrow T \Rightarrow T * F \Rightarrow T * id \Rightarrow F * id \Rightarrow id * id$

8.2 Handle Pruning

A handle is a substring that (a) matches the body of some production, and (b) whose reduction represents one step along the reverse of a rightmost derivation.

Formally: if $S \Rightarrow^*_{rm} \alpha A w \Rightarrow_{rm} \alpha \beta w$, then production $A \rightarrow \beta$, in the position following α , is a handle of $\alpha \beta w$. The string w to the right of the handle must contain ONLY terminals.

Important: the leftmost substring matching a production body is NOT always a handle. Example: In $T * id_2$ (from grammar 4.1), T matches the body of $E \rightarrow T$, but T is NOT a handle here — reducing it would give $E * id_2$, which can never lead to a valid parse.

Handle-pruning process: repeatedly locate the handle in the current right-sentential form and reduce it, until only the start symbol remains — this reconstructs a rightmost derivation in reverse.

Handles during parse of $id_1 * id_2$ (Grammar 4.1)

Right-Sentential Form	Handle	Reducing Production
-----------------------	--------	---------------------

$id_1 * id_2$	id_1	$F \rightarrow id$
$F * id_2$	F	$T \rightarrow F$
$T * id_2$	id_2	$F \rightarrow id$
$T * F$	$T * F$	$E \rightarrow T * F$

Theory Questions — Bottom-Up Parsing & Handles

Q: What is bottom-up parsing? Why is it also called shift-reduce parsing?

Ans: Bottom-up parsing builds the parse tree from the leaves (tokens) up to the root (start symbol). It is called shift-reduce parsing because its two core operations are 'shift' (push input symbol onto stack) and 'reduce' (replace a string on the stack with a nonterminal).

Q: What is a reduction?

Ans: A reduction is the reverse of a derivation step: a substring matching a production's body (on top of the stack) is replaced by the nonterminal at the head of that production.

Q: What is a handle?

Ans: A handle is a substring of the current right-sentential form that (1) matches the body of some production and (2) reducing it corresponds to one step of a rightmost derivation in reverse. The string to the right of a handle contains only terminals.

Q: Why is the leftmost substring matching a production body not always a handle?

Ans: Because reducing it might not lead toward a valid derivation of the start symbol. Example: in $T * id_2$, reducing T (body of $E \rightarrow T$) to E gives $E * id_2$ which is not derivable from E — so T is not a handle there, even though it matches a production body.

Q: Why must the string to the right of a handle contain only terminals?

Ans: Because bottom-up parsing scans left to right and reduces in the order of a rightmost derivation reversed; everything to the right of the current point of interest must already be fully reduced to terminals (not-yet-processed nonterminals cannot exist there).

Q: What is handle pruning?

Ans: The process of successively finding a handle in the current string and reducing it to its production's head, repeated until the string is reduced to just the start symbol — this reconstructs a rightmost derivation in reverse, completing the parse.

Q: Why does an ambiguous grammar mean a right-sentential form can have more than one handle?

Ans: If a grammar is ambiguous, a right-sentential form may have more than one valid rightmost derivation, so there can be more than one handle — this is why we say 'a handle' rather than 'the handle'. If the grammar is unambiguous, every right-sentential form has exactly one handle.

9. Shift-Reduce Parsing — Mechanics

Shift-reduce parsing uses two data structures:

- A STACK — holds grammar symbols (initially empty, marked by \$ at the bottom).
- An INPUT BUFFER — holds the remaining input string to be parsed (marked by \$ at the right end).

Important fact: the handle always eventually appears at the TOP of the stack, never buried inside it. This justifies using a stack for shift-reduce parsing.

9.1 Four Possible Actions

- Shift: push the next input symbol onto the top of the stack.

- Reduce: the right end of the string to be reduced must be at the top of stack; locate the left end within the stack, and replace the matched body with the appropriate nonterminal head.
- Accept: announce successful completion of parsing (stack = \$S and input = \$).
- Error: syntax error detected; call an error-recovery routine.

9.2 General Shift-Reduce Parsing Cycle

Initial configuration: Stack = \$, Input = w\$

The parser shifts zero or more symbols onto the stack until a handle β appears at the top of the stack, then reduces β to the head of the matching production. This cycle (shift...shift...reduce) repeats until either:

- Stack = \$S and Input = \$ $\rightarrow$ ACCEPT (successful parse), or
- An error is detected $\rightarrow$ ERROR.

Key insight: after every reduction, the parser only needs to shift zero or more symbols to bring the NEXT handle to the top of the stack — it never needs to search inside the stack for a handle.

Theory Questions — Shift-Reduce Mechanics

Q: What two data structures does shift-reduce parsing use?

Ans: A stack (holding grammar symbols already processed) and an input buffer (holding the remaining unread input), with \$ marking the bottom of the stack and the end of input.

Q: List and explain the four possible actions of a shift-reduce parser.

Ans: (1) Shift — push next input symbol onto the stack. (2) Reduce — replace a string (handle) on top of the stack with the nonterminal head of the matching production. (3) Accept — announce success when the stack contains just the start symbol and input is empty. (4) Error — call error recovery when no valid shift or reduce action applies.

Q: Why is it justified to always look for the handle only at the top of the stack?

Ans: Because it can be proven (by considering the two possible forms of successive steps in a rightmost derivation) that the handle always eventually appears at the top of the stack, never inside it — so after a reduction, the parser only needs to shift more symbols, never search inside the stack.

Q: When does a shift-reduce parser ACCEPT the input?

Ans: When the stack contains exactly \$ followed by the start symbol (e.g., \$E), and the input contains only \$ (i.e., it's fully consumed).

Q: Why can the parser 'never go into the stack to find the handle'?

Ans: Because both possible cases of successive rightmost-derivation steps show that after reducing a handle to a nonterminal B, the parser shifts zero or more terminals to expose the next handle directly at the top — it is a mathematical guarantee, not a coincidence.

10. Worked Examples — Step-by-Step Simulations

Grammar used for both examples:

$E \rightarrow E + T \mid T \quad T \rightarrow T * F \mid F \quad F \rightarrow (E) \mid id$

Example 1: Parse the string $id_1 * id_2$

Stack	Input	Action
\$	$id_1 * id_2 \$$	Shift
$\$id_1$	$* id_2 \$$	Reduce $F \rightarrow id$
$\$F$	$* id_2 \$$	Reduce $T \rightarrow F$

\$T	* id ₂ \$	Shift
\$T*	id ₂ \$	Shift
\$T*id ₂	\$	Reduce $F \rightarrow id$
\$T*F	\$	Reduce $T \rightarrow T*F$
\$T	\$	Reduce $E \rightarrow T$
\$E	\$	Accept

Final rightmost derivation (in reverse of reductions): $E \Rightarrow T \Rightarrow T*F \Rightarrow T*id \Rightarrow F*id \Rightarrow id*id$.

Example 2: Parse the string (id) + id

Stack	Input	Action
\$	(id) + id \$	Shift
\$ (	id) + id \$	Shift
\$ (id	) + id \$	Reduce $F \rightarrow id$
\$ (F	) + id \$	Reduce $T \rightarrow F$
\$ (T	) + id \$	Reduce $E \rightarrow T$
\$ (E	) + id \$	Shift
\$ (E)	+ id \$	Reduce $F \rightarrow (E)$
\$F	+ id \$	Reduce $T \rightarrow F$
\$T	+ id \$	Reduce $E \rightarrow T$
\$E	+ id \$	Shift
\$E+	id \$	Shift
\$E+id	\$	Reduce $F \rightarrow id$
\$E+F	\$	Reduce $T \rightarrow F$
\$E+T	\$	Reduce $E \rightarrow E+T$
\$E	\$	Accept

Tip for simulation exams: at each step, check if the TOP of the stack matches the RIGHT-HAND SIDE (body) of some production, AND that reducing it leads toward eventually deriving the start symbol. If yes $\rightarrow$ reduce; otherwise $\rightarrow$ shift the next input symbol.

Theory Questions — Simulation Technique

Q: What is the general strategy to decide shift vs reduce while simulating by hand?

Ans: Shift symbols onto the stack until the top of the stack matches the body of a production AND reducing it is consistent with eventually deriving the start symbol from the remaining input (i.e., it is a genuine handle, not just a body match). Then reduce; otherwise keep shifting.

Q: In Example 1, why is T not reduced to E immediately after ' $T \rightarrow F$ ' when the stack is \$T and input is *id₂\$?

Ans: Because at that point T is not a handle — the next input symbol is '*', meaning T is actually part of a larger $T*F$ pattern ($T \rightarrow T*F$ production body in progress). Reducing T to E here would make further parsing impossible (dead end), so the parser shifts '*' instead.

Q: How do you know parsing succeeded?

Ans: When the stack contains just \$ followed by the start symbol (e.g., \$E) and the input contains only \$ — the parser announces 'Accept'.

11. Conflicts During Shift-Reduce Parsing

Some context-free grammars cannot be parsed with shift-reduce parsing because the parser reaches a configuration where — knowing the entire stack and next input symbol — it cannot decide what to do. These grammars are called non-LR grammars. Two kinds of conflicts:

11.1 Shift/Reduce Conflict

The parser cannot decide whether to SHIFT the next symbol or REDUCE a string already on the stack.

Classic example — the Dangling-Else Grammar:

```
stmt → if expr then stmt      | if expr then stmt else stmt      | other
```

In configuration Stack = ...if expr then stmt, Input = else...\$, the parser cannot tell whether 'if expr then stmt' is a complete handle (reduce) or whether it should shift 'else' to build the longer alternative. Resolution: shift/reduce conflict is resolved in favor of SHIFTING — this associates each 'else' with the nearest unmatched 'then', matching normal programmer expectation.

11.2 Reduce/Reduce Conflict

The parser has a handle but the stack contents and lookahead are not enough to decide WHICH production to use for reduction.

Classic example — Procedure Calls vs Array References (both share the same id(...) syntax):

```
(1) stmt → id ( parameter_list ) (2) stmt → expr := expr (3) parameter_list → parameter_list , parameter (4) parameter_list → parameter (5) parameter → id (6) expr → id ( expr_list ) (7) expr → id (8) expr_list → expr_list , expr (9) expr_list → expr
```

For input p(i,j) (token stream id(id,id)), after shifting 'id (id', the parser must reduce the top 'id' but cannot tell whether to use production (5) parameter→id (if p is a procedure) or production (7) expr→id (if p is an array) — the stack alone doesn't reveal p's declared type; symbol-table information is needed.

Solution: modify the lexical analyzer to consult the symbol table and return a distinct token 'procid' for identifiers known to be procedure names — this removes the ambiguity, since the third-from-top stack symbol (id vs procid) then determines the correct reduction, even though it is not itself part of the reduced string.

Theory Questions — Conflicts

Q: What is a shift/reduce conflict? Give an example.

Ans: It occurs when the parser cannot decide whether to shift the next input symbol or reduce a string already on the stack, given the full stack and next input symbol. Example: the dangling-else grammar, where on seeing 'else' after 'if expr then stmt', the parser cannot tell if 'if expr then stmt' should be reduced or if 'else' should be shifted.

Q: What is a reduce/reduce conflict? Give an example.

Ans: It occurs when the parser knows it has a handle but cannot decide which of two or more productions to use to reduce it. Example: distinguishing an array reference from a procedure call when both use identical id(...) syntax — the parser cannot tell whether to reduce id using parameter→id or expr→id without extra (symbol table) information.

Q: Why can an ambiguous grammar never be LR?

Ans: Because LR grammars must allow the parser to uniquely determine the handle (and hence the single correct rightmost derivation) at every step using only the stack and limited lookahead. An ambiguous grammar has more than

one rightmost derivation for some string, so the parser cannot make a unique choice — causing shift/reduce or reduce/reduce conflicts.

Q: How is the dangling-else shift/reduce conflict typically resolved?

Ans: By resolving in favor of shifting the 'else' — this makes the parser always associate an 'else' with the closest previous unmatched 'then', which matches how programmers expect the code to behave.

Q: What does 'k' mean in LR(k)?

Ans: It refers to the number of lookahead symbols of input the parser uses when making a parsing decision. $k=1$ (one symbol lookahead) is the most commonly used in practice; when 'k' is omitted, it is assumed to be 1.

Q: Why do compiling grammars usually fall in the LR(1) class?

Ans: Because LR(1) parsers (using just one token of lookahead) are sufficient to unambiguously parse virtually all practical programming-language constructs while still being efficient to implement via automatic parser generators.

12. Practice Exercises (Textbook Section 4.5.5 Style)

These are worked solutions to the standard exercise style questions on handles and bottom-up parsing — a very common exam/simulation question format.

Exercise A — Identify the Handle

Grammar: $S \rightarrow 0S1 \mid 01$

Right-Sentential Form	Handle	Reducing Production
000111	01 (middle positions 3-4)	$S \rightarrow 01$
00S11	0S1 (positions 2-4)	$S \rightarrow 0S1$

Grammar: $S \rightarrow S S + \mid S S * \mid a$

Right-Sentential Form	Handle	Reducing Production
SSS+a*+	SS+ (positions 2-4)	$S \rightarrow SS+$
SS+a*a+	SS+ (positions 1-3)	$S \rightarrow SS+$
aaa*a++	a (the very first 'a', position 1)	$S \rightarrow a$

Exercise B — Full Bottom-Up (Shift-Reduce) Parses

B1: Parse 000111 using $S \rightarrow 0S1 \mid 01$

Stack	Input	Action
\$	000111 \$	Shift
\$0	00111 \$	Shift
\$00	0111 \$	Shift
\$000	111 \$	Shift
\$0001	11 \$	Reduce $S \rightarrow 01$
\$00S	11 \$	Shift
\$00S1	1 \$	Reduce $S \rightarrow 0S1$
\$0S	1 \$	Shift

\$OS1	\$	Reduce $S \rightarrow OS1$
\$S	\$	Accept

B2: Parse aaa^*a^{++} using $S \rightarrow SS+ \mid SS^* \mid a$

Stack	Input	Action
\$	aaa^*a^{++} \$	Shift
\$a	aa^*a^{++} \$	Reduce $S \rightarrow a$
\$S	aa^*a^{++} \$	Shift
\$Sa	a^*a^{++} \$	Reduce $S \rightarrow a$
\$SS	a^*a^{++} \$	Shift
\$SSa	a^{++} \$	Reduce $S \rightarrow a$
\$SSS	a^{++} \$	Shift
\$SSS*	a^{++} \$	Reduce $S \rightarrow SS^*$
\$SS	a^{++} \$	Shift
\$SSa	a^{++} \$	Reduce $S \rightarrow a$
\$SSS	a^{++} \$	Shift
\$SSS+	a^{++} \$	Reduce $S \rightarrow SS+$
\$SS	a^{++} \$	Shift
\$SS+	a^{++} \$	Reduce $S \rightarrow SS+$
\$S	a^{++} \$	Accept

Note: grammar $S \rightarrow SS+ \mid SS^* \mid a$ generates postfix expressions — $S \rightarrow a$ is a single operand, and $S \rightarrow SS+ / S \rightarrow SS^*$ means 'take the two most recently built operands and apply the operator', matching how postfix (Reverse Polish) notation works.

Theory Questions — Practice Exercises

Q: In grammar $S \rightarrow SS+ \mid SS^* \mid a$, why is a single 'a' always reduced immediately when it appears?

Ans: Because 'a' cannot be part of any longer terminal-matching pattern by itself (the only production with 'a' in its body is $S \rightarrow a$), so there is no benefit to delaying — it is always safe and necessary to reduce it to S right away.

Q: Why is ' $SS+a^{++}$ ' NOT reduced by treating the whole string as one handle?

Ans: Because a handle must exactly match a single production's body; $SS+a^{++}$ as a whole does not match any single production body. Only the correct substring (found by simulating the parse) — here ' $SS+$ ' — is a valid handle at that stage.

Q: What real-world parsing scenario does the grammar $S \rightarrow SS+ \mid SS^* \mid a$ resemble?

Ans: It resembles parsing postfix (Reverse Polish Notation) expressions, where operators come after their operands, and each reduction corresponds to popping two operands and combining them with an operator — exactly like evaluating a postfix expression using a stack.